

Heet: Accelerating Elastic Training in Heterogeneous Deep Learning Clusters

Zizhao Mo
University of Macau
Macau SAR, China
yc17461@um.edu.mo

Huanle Xu *
University of Macau
Macau SAR, China
huanlexu@um.edu.mo

Chengzhong Xu
University of Macau
Macau SAR, China
czxu@um.edu.mo

Abstract

Modern GPU clusters inherently exhibit heterogeneity, encompassing various aspects such as computation and communication. This heterogeneity poses a significant challenge for the elastic scheduling of deep learning workloads. Unfortunately, existing elastic schedulers often overlook the impact of heterogeneity on *scaling efficiency*, resulting in considerably prolonged job completion times.

In this paper, we present Heet, a new Heterogeneity-aware system explicitly developed for elastic training in DL clusters. Heet addresses two critical issues. First, it utilizes a 3-D collaborative filtering method to accurately measure the *scaling efficiency* of all elastic configurations on heterogeneous hosts, substantially reducing profiling overhead. Second, Heet introduces a unique price function to effectively balance *scaling efficiency* and *scheduling efficiency*. Building upon this function, Heet incorporates a scalable mechanism that employs minimum-weight full bipartite matching and opportunistic resource trading to generate dynamic scheduling decisions. Evaluations conducted on cloud clusters and large-scale simulations demonstrate that Heet can reduce job completion time by up to 2.46× compared to existing solutions.

CCS Concepts: • Computer systems organization → Cloud computing.

ACM Reference Format:

Zizhao Mo, Huanle Xu, and Chengzhong Xu. 2024. Heet: Accelerating Elastic Training in Heterogeneous Deep Learning Clusters. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, April 27-May 1, 2024, La Jolla, CA, USA.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASPLOS '24, April 27-May 1, 2024, La Jolla, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0385-0/24/04

<https://doi.org/10.1145/3620665.3640375>

'24), April 27-May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3620665.3640375>

1 Introduction

In recent years, Deep Learning (DL) has achieved remarkable success across various domains. However, training DL models, which entails learning parameters from massive datasets, is often resource-intensive and time-consuming. To expedite this process, it has become common practice to perform DL training on large-scale GPU clusters. Despite this, DL workloads continue to face significant queuing delays as a result of resource contention in production clusters [20, 25, 49]. This emphasizes the crucial significance of efficient job scheduling in accelerating DL training to achieve a minimized job completion time (JCT).

To reduce queuing delay and optimize JCT for DL workloads, the adoption of elastic training techniques has become increasingly prevalent [16, 24, 29, 42]. In contrast to traditional training paradigms, which maintain resources throughout the entire process of a training workload, the elastic mode facilitates dynamic and flexible resource allocation for DL jobs based on their real-time status and cluster contention levels. It should be noted that DL jobs often exhibit diminishing benefits in terms of training throughput as the number of workers increases [16, 24, 29]. Hence, elastic training offers great potential for globally coordinating resource allocation across diverse DL jobs, ultimately enabling the optimal utilization of GPU resources within a cluster.

As the training paradigm transitions to an elastic mode, today's GPU production clusters exhibit a heterogeneous infrastructure comprising diverse devices [8, 27, 36]. This heterogeneity is evident in multiple aspects, including computation and communication variations. For instance, in public clouds like Google Cloud Platform, there exists a significant discrepancy in FP64 computing capabilities among different generations of GPU devices, with differences of up to 48.5 times. Additionally, the rapid advancements in GPUs have introduced another aspect of heterogeneity that is often overlooked: inter-GPU communication. In older-generation accelerators, multiple GPU workers within a DL job must communicate through the same CPU-GPU channel, specifically PCIe. However, this hierarchical channel can lead to a notable reduction in DL training throughput. Thankfully,

the invention of high-bandwidth GPU-oriented interconnects, such as NVLink, has mitigated this issue by reducing communication overhead during training. In popular cloud providers like Google Cloud Platform [3] and Alibaba Cloud [1], clusters consist of a mixture of high-end GPUs with NVLink, high-end GPUs without NVLink, and low-end GPUs with PCIe, further diversifying the hardware configurations.

The execution of elastic DL training in heterogeneous GPU clusters presents several distinct characteristics, primarily attributed to the influence of heterogeneity on scaling efficiency. As delineated in recent studies [16, 24], scaling efficiency refers to the ratio between the training throughput attained through elastic training and that achieved through linear scaling with an equivalent number of workers. A higher efficiency indicates a more resource-efficient approach for scaling up DL training jobs. DL jobs often exhibit superior scaling efficiency on low-end GPUs compared to their high-end counterparts when utilizing the same communication channel. Consequently, excessively prioritizing high-end resources can impede the overall training throughput. Moreover, the introduction of high-end communication channels involves an intricate trade-off: DL workloads have diverse preferences for resources with either high-end communication or computation power. Unfortunately, existing elastic schedulers overlook the scaling efficiency disparity among jobs within the context of heterogeneity. As a result, there is a significant performance degradation in the overall JCT, as evidenced in § 2.3.

In this paper, we aim to develop a new and efficient elastic scheduler that optimizes JCT in heterogeneous clusters by effectively leveraging scaling efficiency. Nonetheless, we face two fundamental challenges in this pursuit. First, the presence of a large number of elastic configurations within heterogeneous environments makes it difficult to comprehensively profile scaling efficiency. Additionally, building analytical models to measure this is generally intractable due to the complex and varying overlap scheme of computation and communication factors across various DL models and training platforms [42]. Second, elastic resource allocation poses a hard trade-off between *scaling efficiency* and *scheduling efficiency* in large-scale clusters. Improving *scheduling efficiency* often requires prioritizing small jobs with short durations [15, 24]. Consequently, directly enhancing one aspect can have adverse effects on the other, ultimately leading to suboptimal results.

To tackle these challenges, we propose Heet, a Heterogeneity aware system for accelerating elastic training in DL clusters. Heet incorporates two essential components. Specifically, it introduces a lightweight profiling solution to effectively reduce overhead while avoiding the need for complex modeling. The key idea revolves around a 3-D collaborative filtering mechanism to learn missing entries from limited information and capture similarities among recurrent DL

training jobs. Building on this profiling, Heet designs a price function for elastic configurations that effectively balances the trade-off between scaling efficiency and scheduling efficiency. Under this function, a smaller price value signifies a higher scheduling priority, enabling Heet to find an efficient schedule by minimizing the overall price. However, directly solving the optimization problem incurs significant computational overhead. To ensure scalability, Heet adopts a two-phase optimization process, starting with a full bipartite matching followed by an opportunistic trading approach. Additionally, Heet's allocation is placement-aware, effectively addressing the straggler effect and limiting the host span for each job.

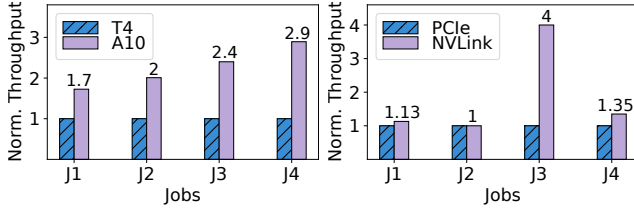
We have implemented a prototype of Heet and conducted comprehensive experiments on both real and simulation testbeds to evaluate its performance. The results demonstrate that Heet effectively reduces the overall JCT by up to 2.46× and makespan by up to 3.96× compared to existing solutions. In summary, this paper presents the following contributions:

- We have conducted an extensive analysis to examine how heterogeneity in modern GPU clusters can impact *scaling efficiency*, emphasizing its critical role in scheduling.
- We have developed a new machine-learning approach that accurately estimates the throughput of elastic configurations with minimal profiling overhead.
- We have designed a scalable scheduling policy that incorporates both *scaling efficiency* and *scheduling efficiency* within heterogeneous clusters.

2 Background and Motivation

2.1 Elastic DL Training

Elastic training has emerged as a major training paradigm for DL workloads in the era of distributed DL training [16, 24, 33, 38–41, 45, 52]. By offering flexible scaling of GPU workers over time, this approach effectively expedites DL training and provides significant resource management advantages for DL clusters. Nonetheless, existing distributed training frameworks, such as TorchElastic and Horovod [40, 45], exhibit diminishing scaling benefits as the number of workers increases, despite the application of advanced techniques [43, 48]. Notably, these endeavors aim to overlap gradient synchronization time by concurrently assessing subsequent gradients. However, the substantial size of the gradient volume often precludes effective concealment within computation time. Furthermore, the overhead of collective communication such as All-Reduce, which is leveraged in the gradient synchronization phase, significantly grows with the number of workers [4]. As a result, *scaling efficiency*, denoting the training benefit of engaging more workers, often displays considerable variability across different jobs and thereby deserves careful investigation [16, 24].



(a) Diverse speedup of single-worker DL jobs (b) Speedup of 4-worker jobs given NVLink over PCIe

Figure 1. The impact of computation and communication heterogeneity on DL training jobs. We use MNasNet (J1), DenseNet (J2), VGG (J3), and ResNet (J4) to represent different DL training jobs.

2.2 Heterogeneity in DL Clusters

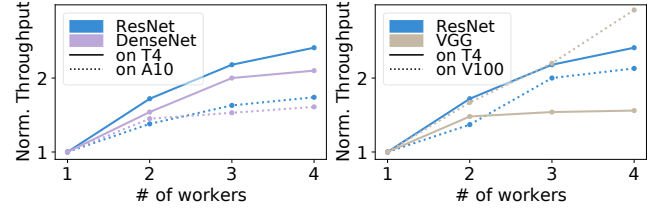
This section will begin by exploring two critical types of heterogeneity found in today’s production clusters: computation and communication heterogeneity. Furthermore, we will elucidate their impact on *scaling efficiency*, a key factor in fine-tuning the overall JCT performance.

2.2.1 Computation heterogeneity. Previous studies [8, 27, 36] have extensively investigated computation heterogeneity, revealing that the speedup of DL training jobs on high-end GPUs varies widely. In Fig. 1(a), we illustrate this by examining the acceleration effects across popular DL training jobs such as MNasNet (J1), DenseNet (J2), VGG (J3), ResNet (J4), etc., by comparing the throughput of various jobs on fast GPU type, i.e., A10, and slow GPU type, i.e., T4, under the single-worker training setting. The results show a significant speedup difference, where ResNet has a $2.9\times$ speedup on A10, while MNasNet is only accelerated by $1.7\times$, highlighting the diverse sensitivity to high-end GPUs.

2.2.2 Communication heterogeneity. Although it is often overlooked, various types of GPU interconnection channels have arisen in data centers. These channels, such as PCIe and NVLink, demonstrate significantly different bandwidths [28]. Nevertheless, due to the difference in model architecture, the reduction of communication overhead for DL training jobs on high-end links also varies widely, resulting in communication heterogeneity. In Fig. 1(b), we investigate the speedup of training throughput for DL workloads using 4 GPUs on a V100 host equipped with NVLink, compared to the same host where NVLink is disabled. The results show that an advanced channel offers no improvement to DenseNet but provides up to $4\times$ speedup to VGG.

2.2.3 Impact of heterogeneity on scaling efficiency. As discussed in § 2.1, the *scaling efficiency* for elastic training is generally a sub-linear function of the number of workers. However, the two types of heterogeneity mentioned above exhibit distinct impacts on this sub-linear behavior.

Computation heterogeneity can degrade scaling efficiency. In heterogeneous clusters, DL training workloads



(a) Training Throughput on T4 and A10 (b) Training Throughput on T4 and V100

Figure 2. The non-uniform *scaling efficiency* of DL jobs in the presence of heterogeneity. The T4 GPU is considered a lower-end option in comparison to the A10 and V100 GPUs, with only the V100 GPUs being connected through NVLink.

on higher-end GPUs exhibit less efficient scaling than those on lower-end GPUs when using the same communication channel. As shown in Fig. 2(a), both selected jobs demonstrate greater *scaling efficiency* on low-end T4 compared to A10 when utilizing the same number of workers. It is worth noting that workers on both hosts are connected only by PCIe. This gap in *scaling efficiency* can be attributed to the greater proportion of communication overhead on high-end GPUs, as the computation process can be completed much faster. Consequently, computation heterogeneity hinders the full utilization of computational power with more high-end workers involved in one DL workload.

Communication heterogeneity necessitates an intricate trade-off to enhance scaling efficiency. Communication heterogeneity can greatly enhance *scaling efficiency*. As exemplified in Fig. 2, the Resnet workload demonstrates significantly higher *scaling efficiency* on V100 with NVLink compared to A10 with PCIe. Therefore, executing a DL workload on a high-end GPU equipped with both powerful computation and communication resources involves an intricate trade-off between reducing computation time and mitigating communication overhead. In particular, when considering a T4 host and a V100 host with high-end NVLink connection and computation power, ResNet exhibits superior *scaling efficiency* on the T4 host, whereas VGG demonstrates better *scaling efficiency* on the V100 host. Additionally, the advantage of scaling up on the V100 host, in comparison to the T4 host, becomes more pronounced as the number of workers increases, as illustrated in Fig. 2(b). The primary factor behind this disparity is that on the V100 host, the substantial reduction in communication overhead for VGG (as evidenced by the $4\times$ speedup with four workers shown in Fig. 1(b)) effectively alleviates the training bottleneck, enabling the full potential of the V100’s advanced computation power to be realized. Conversely, while the ResNet benefits from the advancements in computation power, its *scaling efficiency* is hindered by communication limitations (as indicated by only a $1.35\times$ improvement with NVLink shown in Fig. 1(b)). This ultimately leads to better *scaling efficiency* on the T4 host.

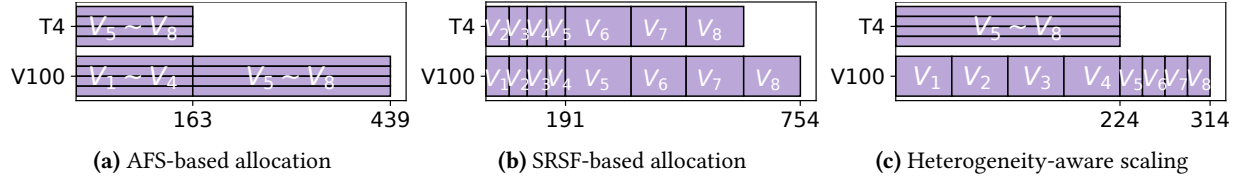


Figure 3. An example illustrating that merely focusing on either scaling efficiency and scheduling efficiency maximization damages the JCT, emphasizing the importance of trade-off between them. The runtime profile of each job is listed in Tab.1.

Table 1. The runtime profiles of jobs vary across different hosts. The notation "T4:1" represents a configuration where only one T4 GPU is allocated

Jobs	Training Time (mins)			
	T4:1	T4:4	V100:1	V100:4
V1-V4	374	245	163	56
V5-V8	1122	735	489	168

2.3 Limitations of Existing Elastic Schedulers

In this section, we demonstrate the insufficiency of existing elastic schedulers in optimizing JCT due to the overlook of *scaling efficiency* within heterogeneous clusters. To illustrate this, we create a scheduling example with eight DL jobs of the same VGG type but diverse lengths are executed within a cluster of two different hosts in Fig. 3. The runtime profile of each job on every host is described in Tab. 1.

AFS. The AFS scheduler [24], is a state-of-the-art solution specifically designed for elastic training. It efficiently allocates idle GPU devices to active DL workloads in an opportunistic manner, minimizing the overall JCT. The core idea is to avoid excessive scaling efficiency degradation among jobs. However, this approach lacks heterogeneity awareness and leads to neglecting the high *scaling efficiency* achieved by allocating more V100 GPUs to VGG. In this scheduling illustration, AFS allocates all jobs with only one GPU, as depicted in Fig. 3(a). Consequently, it forfeits the potential advantage of early completion for short jobs on V100, resulting in diminished scheduling efficiency — evident in a substantial number of concurrently active jobs in the cluster. This approach leads to a total JCT of 2408 minutes.

SRSF. The commonly used SRSF (Shortest Remaining Service First) scheduler [15, 24] allocates most of the advanced resources to small jobs, aiming to facilitate their early completion and reduce the number of active jobs within the cluster. However, in doing so, it fails to consider the low *scaling efficiency* of VGG jobs on T4, which hinders the efficient utilization of GPU resources to advance more jobs simultaneously. As shown in Fig. 3(b), the SRSF scheduler assigns all four workers to VGG jobs on T4, resulting in an overall JCT of 2692 minutes.

Heterogeneity-aware scaling. Upon careful examination of these two schedulers, we have identified that resource allocation, taking into account both *scaling* and *scheduling*

efficiency within a heterogeneous environment, leads to improved scheduling results. As illustrated in Fig. 3(c), this approach entails providing each job with complete access to four V100 GPUs, while restricting scaling with T4. This policy results in a JCT reduction of 1.43× and 1.6× respectively, compared to the AFS and SRSF.

Takeaway. The performance of current elastic schedulers in heterogeneous environments, as measured by JCT, is unsatisfactory, even in straightforward scenarios. This deficiency is chiefly attributed to their inability to discern the diverse scaling efficiencies among different types of GPU devices. Consequently, there exists a critical demand for innovative elastic schedulers capable of adeptly integrating and leveraging scaling efficiency to effectively tackle intricate scheduling scenarios.

3 Heet overview

Heet is a cluster-wide elastic scheduler specifically designed to minimize the overall JCT for DL training workloads in heterogeneous environments. It achieves this goal by making the most use of *scaling efficiency*. In this section, we present the overall architecture of the whole Heet system.

3.1 Design Challenges

Although *scaling efficiency* is critical for optimizing JCT performance, we illustrate in the following that the design of *scaling efficiency*-aware elastic schedulers presents several challenges in heterogeneous environments.

C1: Large search space of elastic configurations in heterogeneous environments. Efficiently measuring the *scaling efficiency* across all potential elastic configurations is non-trivial. One possible approach is to employ lightweight profiling techniques and construct quantitative models capable of inferring the training performance for each new elastic configuration by explicitly modeling factors such as communication and computation. However, it should be noted that this approach becomes generally intractable due to the complex and varying overlap scheme of these elements across various DL models and training platforms [42]. In order to ensure wider applicability, we are exploring alternative approaches that directly sample training throughput across different elastic configurations. However, this approach can easily lead to significant profiling overhead due to the extensive search space of candidate configurations. Therefore, it

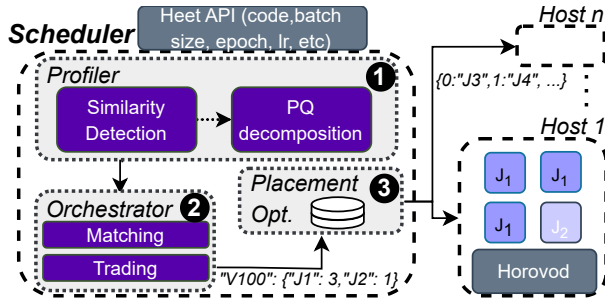


Figure 4. The system architecture of Heet.

is crucial to find effective solutions that can reduce profiling overhead while avoiding the need for complex modeling.

C2: Challenging trade-off between scheduling efficiency and scaling efficiency. In addition to *scaling efficiency*, it is crucial to also achieve high scheduling efficiency by effectively prioritizing short jobs, recognizing their importance in reducing the number of active jobs, as demonstrated in § 2.3. However, optimizing both factors poses a challenging task within a large-scale cluster, given the coexistence of diverse job types and accelerators with significantly varied performance levels. Specifically, improving scaling efficiency often leads to increased job length since it may require allocating more low-end GPU devices to a DL job, resulting in longer training times. Conversely, increasing scheduling efficiency generally involves allocating more advanced GPUs to a specific job, which can adversely affect scaling efficiency. As a consequence, Heet needs to carefully address the hard trade-off between scaling and scheduling efficiency. Through this strategic approach, Heet has the potential to enhance the overall JCT by up to 1.74× in comparison to policies that solely optimize one aspect, as evidenced in § 6.5.

3.2 Architecture & Workflow

The complete system architecture of Heet, along with the scheduling workflow, is illustrated in Fig. 4. DL developers can utilize the API provided by Heet to submit their DL workloads to a GPU cluster.

Upon the submission of new DL workloads, the Profiler module (shown as ❶ in Fig. 4) initiates the profiling process for each job. To ensure optimal resource orchestration, Heet needs to gather comprehensive throughput information for each job, considering different numbers of GPU workers on different host types. However, this process can be prohibitively expensive, as highlighted in C1. To overcome this challenge, Heet incorporates a throughput estimation mechanism based on collaborative filtering. This mechanism allows Heet to efficiently retrieve accurate throughput information for each job with minimal-level profiling, avoiding the need for extensive and resource-intensive profiling.

To facilitate cluster-wide scheduling, we have developed an Orchestrator module (represented as ❷ in Fig. 4) in Heet.

This module is responsible for generating scheduling decisions by utilizing throughput information from job conditions, information provided through the API, and real-time job progress. Our design enables Heet to perform a two-phase optimization process, which includes matching and trading, to jointly optimize *scheduling efficiency* and *scaling efficiency*, thus addressing the challenge detailed in C2.

Furthermore, we have introduced a Placement Optimizer module (depicted as ❸ in Fig. 4) to enhance the placement process by incorporating the popular framework Horovod [45]. Horovod enables dynamic scaling of workers during DL training without the need for restarts or resuming from saved checkpoints. Specifically, this module allocates previously utilized workers to jobs, to leverage the capabilities of Horovod for cost-effective reconfiguration.

4 Design of Heet

This section furnishes a thorough and detailed account of the design for each module within Heet. We initiate an investigation into a new 3-D collaborative filtering approach, skillfully striking a balance between profiling accuracy and profiling overhead. Subsequently, we delve into the specifics of the scheduling and placement scheme, leveraging the information gleaned from the profiling process.

4.1 Profiling Elastic Configurations

4.1.1 Profiling methodology. The training throughput of each DL job is influenced by various factors, such as communication, computation, and the overlap scheme between them. Moreover, the actual computation time is significantly impacted by computation heterogeneity, while communication time is closely related to communication heterogeneity. Additionally, the degree of overlap between these two elements is heavily influenced by the training platform. Consequently, constructing precise models to directly quantify these factors becomes extremely difficult [42]. Consequently, we turn to the use of machine learning methods that can circumvent the need for complex modeling. Specifically, we employ collaborative filtering [19, 47], an effective method used in recommendation systems, to estimate the missing runtime profile for incoming jobs by combining known information from minimal-level profiling.

In the literature, various methods employing collaborative filtering techniques have been proposed to recover missing information in cluster management scenarios [10–12]. However, our specific problem involves throughput information (analogous to ratings in the Netflix Challenge [6]) that is jointly determined by three dimensions: job type, host type, and number of workers. The introduction of varying numbers of workers for each job renders the aforementioned methods, which typically only consider two other dimensions, inapplicable to our setting. As a result, we introduce a pioneering 3-D collaborative filtering method to estimate DL

training within the context of heterogeneity. In particular, Heet applies PQ-decomposition [7, 50], a popular SVD-based method. It learns the missing entries for the corresponding throughput matrix of each job, where rows and columns represent the host type and number of GPU workers, respectively. By leveraging PQ-decomposition, Heet can capture latent factors such as the impact of communication and computation heterogeneity on training through the factorization of the sparse matrix. Consequently, Heet automatically generates missing throughput information in a heterogeneity-aware manner. Moreover, in DL workloads, it is common to observe comparable training behavior across different hosts. For instance, hyperparameter tuning, an essential task for tenants, often involves utilizing the same neural network with minor variations in parameters such as learning rate and batch size. To further reduce the profiling overhead, Heet also leverages the similarity between the newly submitted job and those previously processed to recover missing entries by referencing the already profiled information from the past.

4.1.2 Fast and accurate profiling. Heet devises a minimum level profiling scheme for each DL job within the clusters to strike a balance between estimation accuracy and profiling overhead. This scheme involves profiling a limited number of times per job, effectively capturing the influence of heterogeneity on *scaling efficiency*. In particular, we gather a single sample on each host type, as illustrated below:

$$\Theta_j = \begin{pmatrix} 1 & 2 & 3 & 4 \\ & \theta_{j,1,2} & & \\ & & \theta_{j,2,3} & \\ & & & \theta_{j,3,4} \end{pmatrix} \begin{matrix} \text{type1} \\ \text{type2} \\ \text{type3} \end{matrix},$$

where $\theta_{j,i,k}$ denotes the throughput of job j on host type i given k workers. Each sample collection only requires a mini-batch iteration time. As a result, the profiling overhead is restricted to $O(\min\{M, N\})$ where M denote the maximum worker size and N is the number of host types.

For jobs that exhibit high similarity, their values in the throughput matrix should be nearly identical for corresponding entries. Consequently, Heet evaluates similarity by calculating the average of $\min\{M, N\}$ error rates, where a smaller value indicates higher similarity. If there are jobs with error rates below the specified threshold of 0.1, Heet utilizes the average throughput of these fully profiled jobs as the prediction. In cases where such jobs do not exist, Heet employs SVD-based PQ-decomposition to populate Θ_j [2]. This process involves decomposing Θ_j into $P_{M \times r}$ and $Q_{r \times N}^T$ via minimizing the regularized squared error, where r is the dimension of the latent space and the loss function is formulated as:

$$\sum_{i,j,k} (\theta_{j,i,k} - \hat{\theta}_{j,i,k})^2 + \lambda(b_{j,k}^2 + b_{j,i}^2 + \|Q_{j,k}\|^2 + \|P_{j,i}\|^2). \quad (1)$$

In this context, the predicted throughput is given by $\hat{\theta}_{j,i,k} = \mu_j + b_{j,i} + b_{j,k} + Q_{j,k}^T P_{j,i}$, where μ_j represents the average value, $b_{j,i}$ and $b_{j,k}$ are bias, and $Q_{j,k}$ and $P_{j,i}$ are matrix factors. Additionally, λ corresponds to the regularization coefficient. The optimization process utilizes the conventional stochastic gradient descent method elaborated in [23], where parameters are updated as:

$$\begin{aligned} b_{j,k} &= b_{j,k} + \gamma(\theta_{j,i,k} - \hat{\theta}_{j,i,k} - \lambda b_{j,k}), \\ b_{j,i} &= b_{j,i} + \gamma(\theta_{j,i,k} - \hat{\theta}_{j,i,k} - \lambda b_{j,i}), \\ P_{j,i} &= P_{j,i} + \gamma(Q_{j,k}(\theta_{j,i,k} - \hat{\theta}_{j,i,k}) - \lambda P_{j,i}), \\ Q_{j,k} &= Q_{j,k} + \gamma(P_{j,i}(\theta_{j,i,k} - \hat{\theta}_{j,i,k}) - \lambda Q_{j,k}). \end{aligned}$$

Here, γ represents the step size. To enhance the accuracy of PQ-decomposition, the throughput on each host type is normalized based on that achieved by single-worker training. This normalization facilitates the identification of relative scaling trends for each host type. Consequently, for jobs of low similarity with previously observed jobs, a total of $(M + N)$ samples are collected.

Remarkably, Heet's 3-D collaborative filtering method adeptly strikes a delicate balance between accuracy and complexity. As elucidated in § 6.2, we illustrate that Heet attains an optimal equilibrium, thereby enhancing both the scheduling outcomes and its applicability to production environments simultaneously.

4.2 Cluster-level Orchestration

The objective of Heet's Orchestrator is to dynamically allocate heterogeneous GPU devices to each DL workload in an elastic manner, with the aim of minimizing the overall JCT.

4.2.1 Basic scheduling model. To illustrate the scheduling mechanism employed by Heet, we first introduce the fundamental scheduling model.

We define the elastic DL scheduling problem within heterogeneous clusters as follows: The cluster is composed of N types of physical hosts, with each host type i equipped with T_i GPU devices. Each specific host carries a number of GPUs that is a power of two. A DL training job j can be submitted to the cluster at any time. When job j is submitted, the Orchestrator receives information regarding its training throughput across all available elastic configurations from the Profiler. With this information, the Orchestrator determines the elastic allocation vector $\vec{x}_j(t) = \{x_j^1(t), x_j^2(t), \dots, x_j^N(t)\}$ at each time t , where $x_j^i(t)$ denotes the number of GPU workers assigned to job j from host type i . To ensure that the allocation remains within the limits of the entire cluster's capacity, the Orchestrator must adhere to the following constraint:

$$\sum_j x_j^i(t) \leq T_i, \quad x_j^i(t) \in \mathbb{N}, \quad \forall 1 \leq i \leq N, \forall t. \quad (2)$$

Furthermore, to address the issue of the straggler effect [9, 30, 31] where the slowest worker can hinder DL training

speed and waste the computation power of high-end GPUs, Heet ensures that all workers within each job are allocated exclusively to the same host type, introducing the following norm constraint:

$$\|\vec{x}_j(t)\|_0 \leq 1, \quad \forall j, \forall t. \quad (3)$$

Problem hardness. It can be easily demonstrated that the scheduling problem at hand is NP-hard, as it poses a greater challenge than the conventional NP-hard elastic scheduling problem in a homogeneous environment [24, 34]. Additionally, given that DL jobs arrive dynamically in the cluster, effective scheduling must adaptively respond to this dynamic nature without knowing future arrivals. Consequently, finding the optimal solution for this problem is inherently intractable, leading us to rely on efficient heuristic solutions.

4.2.2 Scheduling price design. An efficient heuristic solution must achieve a delicate balance between *scheduling efficiency* and *scaling efficiency*. The former involves assigning priority to small jobs, aiming to minimize the number of active jobs within the cluster at any given time. Conversely, the latter requires prioritizing jobs that yield the greatest scaling benefit, enhancing the overall productivity of the system. To address this challenging trade-off, Heet designs a crucial metric called the "scheduling price", which is defined as the ratio between *job length* and *scaling efficiency*. In this context, *job length* refers to a timely metric that measures the expected time required to complete a job j given $x_j^i(t)$ GPUs from hosts of type i . Specifically, $\text{len}_j^i(x_j^i(t)) = r_j / f_j^i(x_j^i(t))$, where $f_j^i(\cdot)$ captures the throughput on host type i under a given configuration, and r_j represents the training iterations required of j . Moreover, *scaling efficiency* quantifies the change in throughput when given $x_j^i(t)$ workers of resource type i compared to the throughput achieved under linear scaling. It can be formulated as: $s_j^i(x_j^i(t)) = f_j^i(x_j^i(t)) / (x_j^i(t) \cdot f_j^i(1))$. As a result, the scheduling price p_j^i is a function of $x_j^i(t)$ taking the form of:

$$p_j^i(x_j^i(t)) = \frac{\text{len}_j^i(x_j^i(t))}{s_j^i(x_j^i(t))} = \frac{r_j \cdot f_j^i(1) \cdot x_j^i(t)}{(f_j^i(x_j^i(t)))^2}. \quad (4)$$

By incorporating this new definition of scheduling price, we are able to transform the original scheduling problem into a problem of minimizing the overall price. It is worth noting that in many scenarios, the value of r_j may not be available upon job submission. However, Heet is still capable of handling such jobs by considering only the *scaling efficiency* in the price function during the optimization process. In this scenario, the optimization is performed among jobs that have obtained the smallest amount of services thus far, which serves as a good approximation to the SRSF [17, 24].

4.2.3 Partition based min-price matching. Minimizing the overall price in Eq. (4), while satisfying the constraints in Eq. (2) and (3), poses a nonlinear integer programming

problem. However, applying existing optimization solvers to tackle this problem is not feasible due to the absence of a closed-form expression for the throughput function $f_j^i(\cdot)$. Moreover, attempting to exhaustively enumerate all feasible solutions in order to find the optimal solution would lead to overwhelming computational complexity. This is due to the vast number of feasible solutions, which can be estimated as follows:

$$\max_i \left(\frac{T_i + \mathbb{J}(t) - 1}{\mathbb{J}(t) - 1} \right) \geq \max_i \left(\frac{T_i + \mathbb{J}(t) - 1}{e} \right)^{T_i}. \quad (5)$$

In the inequation above, $\mathbb{J}(t)$ represents the total number of active DL jobs at time t , and the inequality stems from Stirling's formula [44]. When $\max_i T_i = 10$ and $\mathbb{J}(t) = 10$, the estimated number of feasible solutions exceeds 3×10^9 .

To address this challenge, Heet employs a novel approach known as "resource partitioning". This technique evenly partitions resources in the cluster and allocates them among DL jobs based on the unit of partition, which significantly reduces the computational complexity. Specifically, the partition size is determined by $w(t) = \lceil \sum_i T_i / \mathbb{J}(t) \rceil$. With this design, the partition size adapts dynamically to the resource contention within the cluster, effectively preventing the allocation of either too few or too many GPUs to a single DL job. As a result, this partitioning strategy helps strike a better balance between *scheduling efficiency* and *scaling efficiency*.

The power of resource partitioning also extends to addressing placement issues inherent in DL training. To achieve this, Heet enhances the partition size design by adopting a buddy-allocation: $w(t) = \text{round} \left(\min \{ \sum_i T_i / \mathbb{J}(t), c \} \right)$, where the result is rounded to the nearest value of $2^n, n \in \mathbb{N}$. This rounding adopts a similar idea to previous works such as [16, 24, 55]. In this context, the constant c limits the maximum number of GPUs assigned to a DL training job. Through meticulous value selection and the allocation of *at most one* resource partition to each DL job, Heet adeptly tackles placement concerns by mitigating network congestion resulting from cross-network communication. The subsequent theorem provides further insights into the selection procedure, and the detailed proof can be found in [5].

Theorem 1. *By setting c as the common divisor of T_i for all i , Heet eliminates stragglers and limits the host span to c/h , where h represents the minimum number of GPUs on a host.*

Leveraging the power of resource partitioning, Heet transforms the goal of minimizing overall prices into a min-weight full bipartite matching problem. This transformation ensures that each job is assigned a maximum of one resource partition and each partition is allocated to at most one job, thereby automatically satisfying the constraint stated in Eq. (2) and (3) respectively. To achieve this, Heet constructs a weighted bipartite graph denoted as $G = ((U, V), E)$, where U , V , and E represent the active jobs, resource partitions, and all possible job-resource pairs, respectively. Notably, each job-resource

pair $e \in E$ is assigned a weight $\omega(e)$ that corresponds to the scheduling price $p_j^i(w(t))$. The objective is to schedule as many jobs as possible while simultaneously reducing the overall scheduling price. To accomplish this, the Orchestrator employs Karp's algorithm [26], resulting in a min-weight full matching $M \subseteq E$, where $|M| = \min(|U|, |V|)$. In cases where the mechanism leaves some workers idle, the Orchestrator assigns them greedily to jobs with high *scaling efficiency* after the matching process.

As explained in § 4.2.2, when the training workload r_j is unavailable to the Orchestrator, Heet resorts to a traditional sharing scheme that schedules jobs in rotation. This scheme involves maintaining a counter to track the allocated time for each job. During scheduling, Heet chooses only $\lceil \sum_i T_i / w(t) \rceil$ jobs with the smallest count value. Following that, the same matching process as described earlier is applied.

4.2.4 Opportunistic trading. The matching solution proposed above, although effective in many cases, can lead to sub-optimality in complex scenarios due to the specific design requirement of having the same worker size for each partition. To overcome this limitation, we introduce a second-phase opportunistic trading mechanism to enhance the system. The mechanism identifies opportunities to improve *scaling efficiency* without causing any delays in the JCT of existing jobs by iteratively reallocating resources among jobs. It involves trading between the most scaling-inefficient job and the one that stands to benefit the most at each iteration, effectively optimizing the scheduling process.

However, finding the optimal trading strategy poses significant challenges due to the resource heterogeneity and potential variability in the number of workers starved jobs can receive from all selected jobs during the matching phase. This results in high computational complexity. To tackle this challenge, we introduce a restriction during the trading phase, allowing starved jobs to acquire only one worker. This restriction enables more efficient utilization of workers from scaling-inefficient jobs. To illustrate this concept, we develop a trading heuristic inspired by the comparison of JCT in a two-job scenario, as depicted in Fig. 5. In this example, there is only one resource partition k with $w(t)$ workers, and J_1 is determined to be scheduled first by the matching phase ahead of J_2 as shown in Fig. 5(a). The overall JCT is

$$2 \cdot \text{len}_{J_1} / f_{J_1}^k(w(t)) + \text{len}_{J_2} / f_{J_2}^k(w(t)). \quad (6)$$

If J_1 is deemed scaling-inefficient, one worker could be turned over to J_2 , resulting in an overall JCT of

$$\frac{2 \cdot \text{len}_{J_1}}{f_{J_1}^k(w(t) - 1)} + \frac{\text{len}_{J_2} - \text{len}_{J_1} \cdot f_{J_2}^k(1) / f_{J_1}^k(w(t) - 1)}{f_{J_2}^k(w(t))}. \quad (7)$$

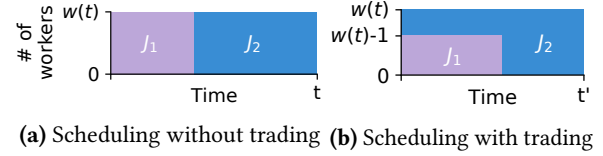


Figure 5. An illustrative case highlighting the advantages of trading.

Heet determines whether such a trade should be executed by evaluating the difference between (6) and (7):

$$2 \cdot \frac{f_{J_1}^k(w(t)) - f_{J_1}^k(w(t) - 1)}{f_{J_1}^k(w(t))} \leq \frac{f_{J_2}^k(1)}{f_{J_2}^k(w(t))}. \quad (8)$$

To generalize this two-job case to multi-job scheduling in large-scale production clusters, Heet repeats the trading process between job J_1 and a starved job J_2 . Heet selects J_1 based on the smallest value of $2 \cdot (f_{J_1}^k(n) - f_{J_1}^k(n - 1)) / f_{J_1}^k(n)$, where $n \in [2, w(t)]$, for its share k . Additionally, Heet chooses J_2 based on the maximum value of $f_{J_2}^k(1) / f_{J_2}^k(w(t))$, utilizing one worker of resource type k .

4.3 Optimizing Worker Placement

In addition to the orchestration decisions made by the Orchestrator module, Heet incorporates a Placement Optimizer module to further enhance the job placement process by considering the previous locations of the jobs. The primary objective is to minimize the re-configuration overhead that arises from changes in the number of GPU workers for DL jobs. Specifically, as the re-configuration of multi-worker jobs is more expensive compared to the single-worker ones due to the need for communication group re-initialization and parameter synchronization among workers, Heet gives top placement priority to multi-worker jobs with unchanged resource demands to keep their workers invariant. The same strategy applies to optimize the placement of single-worker jobs. It is worth noting that Heet's Placement Optimizer focuses solely on optimizing the worker locations of jobs without making secondary changes to the numerical allocation result, i.e., how many workers are assigned to each job. This sets it apart from existing approaches, such as AFS and ElasticFlow [16, 24].

5 Implementation Details

We have implemented a prototype system for Heet using Python. The prototype system runs on Linux and provides support for DL training jobs written in PyTorch [40]. In order for Heet to operate correctly, we implement the exchange of control messages between the Scheduler and Agent modules on each host through a Restful API. Additionally, checkpoint files are seamlessly transferred among hosts using rsync, ensuring a user-imperceptible process.

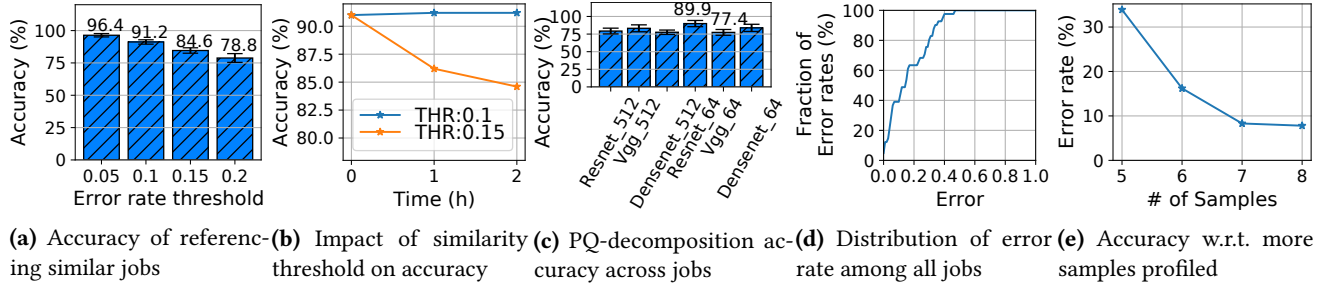


Figure 6. Profiling accuracy. (a-b): Results from referencing similar jobs. (c-e): Results from PQ-decomposition.

Profiling. In Heet, each profiling task is only a short run using a few batches since the training time for different batches under each configuration is almost the same. Tasks for single-worker profiling on each host type are granted the highest priority during the profiling phase. When the demanded throughput is unavailable in the orchestration phase, we estimate the multi-worker throughput by averaging the gain from historic profiled jobs with an equal number of workers on the corresponding host type. Moreover, we leverage *Surprise* [23] to achieve efficient SVD-based decomposition. Specifically, we adopt the default value provided by *Surprise* for $n_factors$, γ , and λ .

Matching. We employ NetworkX [18] to determine the optimal matching. Specifically, we designate the job and resource partitions as two distinct types of nodes, and connect these two node groups using undirected weighted edges.

Elasticity. To dynamically adjust the allocation of job shares, we make use of an adaptable *host discovery function* in Horovod framework, which serves as an interface for performing elastic expansions or contractions of workers. As this interface only allows us to specify the number of workers, we also implement a mapping component within each host. This component enables the translation of the *local ranks* of each job in Horovod to physical workers.

Co-scheduling non-elastic jobs. To address the inherent unsuitability of scaling for certain DL jobs, we develop an extension to Heet. This extension seamlessly co-schedules these jobs with elastic ones. Specifically, we replace the term $f_j^k(w(t))$ for such jobs with $f_j^k(1)$. During the matching phase, if non-elastic jobs are selected, they are given priority to release unused workers to other jobs, taking into consideration the left part of Eq. (8). Similarly, if these non-elastic jobs are not selected during the matching phase, they still receive top priority during the trading process due to the significant value in the right part of Eq. (8).

6 Evaluation

In this section, we present a comprehensive evaluation of Heet, assessing its performance against existing scheduling policies through deployment and large-scale simulations.

6.1 Evaluation setup

Cluster setup. We used a 24-GPU cluster in our experiments on the Alibaba cloud platform. The cluster consists of three types of servers, each with four GPUs. The first type has T4 GPUs interconnected by PCIe, the second type has A10 GPUs interconnected by PCIe, and the third type has V100 GPUs interconnected by NVLink. We have two servers for each server type in our cluster configuration. All DL training tasks are executed using PyTorch 1.13 with CUDA 11.7 and NCCL 2.14.3. To facilitate elastic resource re-configuration, we utilized Horovod v0.26.0. Additionally, considering the 10Gbps network capacity within the cluster, we set the value of c to 8. This setting enables us to control network packing and effectively eliminate any adverse impact of stragglers.

Workload. The job arrival process follows the Microsoft production trace [25]. Specifically, we referenced the submission time, elapsed time, and requested worker size for each job. Moreover, we filled the vacant workload type information in the trace by selecting popular tasks listed in Tab. 2. To thoroughly examine the performance, we utilized two traces: Trace A, which consists of 800 jobs and represents scenarios with moderate contention, and Trace B, which comprises 1500 jobs and captures scenarios with heavy contention. Both traces record job arrivals in three days.

Table 2. The DL workloads used in our evaluation with their specified dataset.

Model	Task	Dataset
ResNet	CV	ImageNet
MNasNet	CV	ImageNet
VGG	CV	Cifar-100
DenseNet	CV	Cifar-100
LSTM	NLP	Wikitext-2
RNN	NLP	Wikitext-2
Transformer	NLP	CoLA
Transformer	NLP	Multi30k
Deepspeech2	Speech	LibriSpeech

Baselines. In order to demonstrate the advantages of Heet, we included a wide range of baselines. Specifically, we incorporated two elastic schedulers striving for JCT minimization

on the length-aware setting, AFS-L [24] and SRSF. Additionally, we compared Heet with Gavel [36], a heterogeneity-aware scheduler. Furthermore, to demonstrate the advancements of Heet in the length-oblivious setting, we introduced three popular schedulers: Tiresias-L [17], AFS-P [24], and Max-Min [15]. This diverse set of baselines allows us to highlight the superior performance of Heet. As we evaluate the performance in heterogeneous clusters, we have adapted the heterogeneity-oblivious schedulers to operate effectively in our heterogeneous environments. This is achieved by establishing a separate queue for each server type. When new jobs arrive, they are randomly assigned to one of the queues. **Metrics.** To comprehensively evaluate the effectiveness of Heet, we examined its performance from both macroscopic and microscopic perspectives. We analyzed the overall JCT and makespan to provide a macroscopic view of Heet's effectiveness. We also assessed the average *scaling efficiency* (SE) and blocking index (BI) to offer a microscopic view of its advantages. More specifically, we adhere to the definitions of SE and BI as outlined in AFS [24]:

$$SE = \sum_{j \in J_R(t)} \frac{f_j^i(x_j^i(t))}{|J_R(t)| x_j^i(t) f_j^i(1)}; BI = \sum_{j \in J_P(t)} \frac{\phi_j(t)}{|J_P(t)| \min_i \text{len}_j^i(1)}.$$

In this context, $J_R(t)$ and $J_P(t)$ represent the set of running jobs and pending jobs at time t . Moreover, $\phi_j(t)$ denotes the fraction of pending time for job j from its arrival until time t . With this definition, a poorly suited scaling up results in a low SE value, while short jobs experiencing prolonged starvation contribute to a high BE value.

6.2 Profiling Accuracy

In this part, we present the evaluation of the profiling module in Heet via conducting independent assessments of two profiling components: job similarity identification and PQ-decomposition. We gathered training throughput data from 8 different workload types as the ground truth in our cluster. We then compared the predictions made by Heet with the actual ground truth to quantify the error rates.

Accuracy of referencing similar jobs. Heet utilizes a strategy of reusing profiling results from similar jobs. We demonstrate the efficiency of this strategy by assessing the impact of adjusting the similarity threshold on prediction accuracy. In Fig. 6(a), we observe a decline in accuracy as the threshold value increases. A higher threshold includes more similar jobs, which introduces more noise to the prediction process. To confine the reference scope, Heet sets the threshold at 0.1, resulting in an accuracy of 91.2% for newly arrived jobs. This threshold strikes a balance between the availability of historical job data and prediction accuracy. Furthermore, this threshold prevents on-the-fly accuracy degradation. To demonstrate this claim, we investigated a scenario where jobs of the same workload type but different batch sizes continuously enter our cluster. As shown in Fig. 6(b), when the

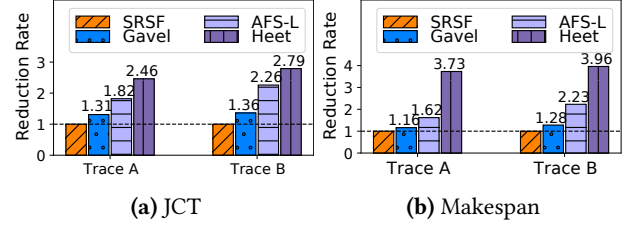


Figure 7. JCT and makespan performance in clusters where job length is provided. The baseline is SRSF.

threshold is set at 0.1 and 0.15 initially, with only a few jobs being referenced, a job can achieve an accuracy of nearly 91%. However, as time progresses and more jobs are recognized as highly similar, the accuracy under a threshold of 0.15 drops to approximately 85%, while the accuracy under a threshold of 0.1 remains almost constant over time.

Accuracy of PQ-decomposition. We analyze the accuracy of estimation using PQ-decomposition by examining several representative DL workloads under various runtime configurations, such as different learning rates and batch sizes. As shown in Fig. 6(c), their accuracy ranges from 77.4% to 89.9% due to inherent differences in scaling and computation across diverse machine types. Furthermore, we present the overall distribution of error rates in Figure 6(d). The distribution reveals that over 60% of entries have an error rate lower than 20%, and the median error rate value among entries is 17% under our scheme.

Accuracy v.s. profiling overhead. In Fig. 6(e), we observe a trade-off between estimation accuracy and profiling overhead by adjusting the number of samples collected for PQ-decomposition. With only five samples, Heet fails to provide precise estimates due to the limited sample size, resulting in an error rate of nearly 32%. Conversely, when eight samples are used, the accuracy improves, and the error rate reduces to nearly 8%. However, increasing the number of samples beyond the default value of Heet (6 in our cluster setting) offers only marginal benefits while incurring heavier overhead.

6.3 End-to-End Evaluation

We have assessed the scheduling performance of Heet in our GPU cluster compared to other schedulers under different settings, focusing on two crucial metrics: JCT and Makespan.

Evaluation on length-aware setting. In Fig. 7, we illustrate the reduction rates of JCT and makespan, under the scenario where the number of training iterations for each job is known. Since SRSF neglects *scaling efficiency*, it causes severe HOL blocking of long jobs. Gavel, which allocates resources based on user-specified worker size, exhibits slightly better JCT and makespan performance than SRSF due to reduced scaling inefficiency. However, Gavel lacks the flexibility to adjust worker groups for individual jobs, hindering fast completion and starvation mitigation. AFS-L performs closest to Heet, as it considers *scaling efficiency* as well. Nonetheless, AFS-L falls

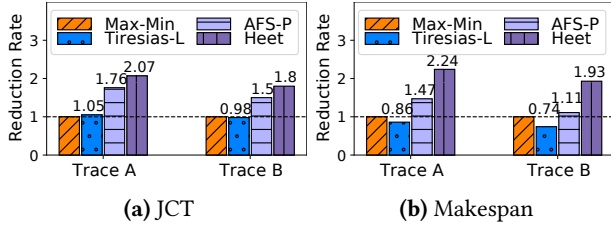


Figure 8. JCT and makespan performance in clusters where job length is not provided. The baseline is Max-Min.

short in maximizing both scaling and *scheduling efficiency* in heterogeneous settings, resulting in up to 35% longer JCT and up to 2.3 times longer makespan compared to Heet. Importantly, Heet consistently outperforms the baselines in both moderate and heavy contention scenarios.

Evaluation on length-oblivious setting. We further investigated the scheduling performance in a scenario where the number of training iterations is unknown. To serve as baselines, we considered Max-Min, Tiresias-L (2-D Least Attained Service), and AFS-P. In Fig. 8, we observe that both Max-Min and Tiresias-L exhibit disadvantages in terms of JCT and makespan. This is because they allocate resources to jobs based on user-specified worker size, overlooking potential improvements in *scaling efficiency*. AFS-P, on the other hand, shows some advantages in length-oblivious scheduling. However, its failure to prioritize jobs already assigned one worker results in limited *scaling efficiency* improvement. Unsurprisingly, Heet surpasses all length-oblivious baselines by capturing and maximizing *scaling efficiency* in the presence of heterogeneity. Specifically, Heet outperforms baselines by up to 2.07× and 2.6×, in terms of JCT and makespan respectively.

Evaluation on co-scheduling non-elastic jobs. We continued with the evaluation of scheduling performance in clusters where both single-worker and elastic jobs are presented. In particular, we specifically modified the workload type of 30% of the jobs in trace A and trace B to RNN-based models. These jobs exclusively utilize a single worker during training. Heet demonstrates substantial enhancements over the baselines in this scenario, achieving a JCT improvement of up to 2.09× and a makespan reduction of up to 3.53×.

Training accuracy. As Heet does not alter the global batch size of jobs across various configurations, it sustains the statistical efficiency specified by users over time. To validate this consistency, we evaluated the accuracy of all jobs. The results highlight that Heet demonstrates a minimal accuracy difference, reaching at most 0.8%, when compared to training that utilizes a single worker throughout its entire lifespan.

6.4 Re-configuration Overhead of Elastic Training

We provide a detailed analysis of re-configuration overhead for specific DL training jobs, namely VGG19, ResNet152, and Densenet121, in Fig. 10. The scale-up operation involves

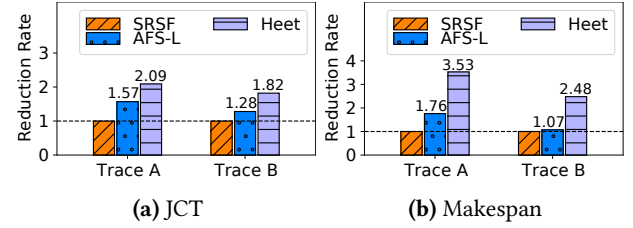


Figure 9. JCT and makespan performance in clusters where elastic and single-worker jobs co-exist in the cluster. The baseline is SRSF.

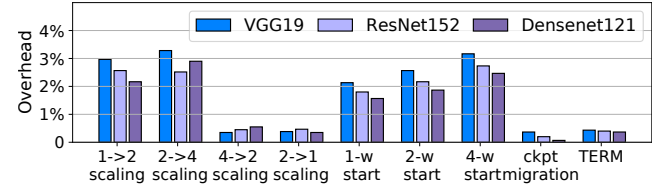


Figure 10. Re-configuration event overhead. "1->2" denotes scaling up workers from one to two, and "1-w start" means the start overhead of one-worker jobs. The length of the scheduling round is 300s.

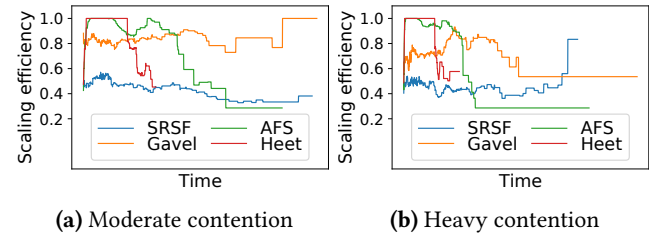


Figure 11. *Scaling efficiency* of various schedulers across different resource contention levels.

communication group re-initialization and parameter synchronization, resulting in an average time cost of approximately 2.5%, which is about 5× more than that of scale-down events. Although adopting elastic scale-up only slightly reduces overhead compared to the combined termination and start overhead, Heet's elastic scale-down operation proves highly efficient, reducing overhead by up to 6× compared to the counterpart that necessitates stop-and-restart operations. Overall, the re-configuration overhead remains marginal relative to the scheduling interval, reaching only 3%.

6.5 Microscopic Evaluation: An In-depth Analysis

To obtain a detailed understanding of Heet's superiority, we compared Heet and other baseline methods utilizing a large-scale simulation testbed, focusing on micro metrics. Our testbed consists of a cluster with 120 GPUs, distributing 40 GPUs across each machine type. To ensure accurate results, we incorporated real training throughput information obtained from various deployment conditions for each training job type, derived from actual execution in the cluster testbed.

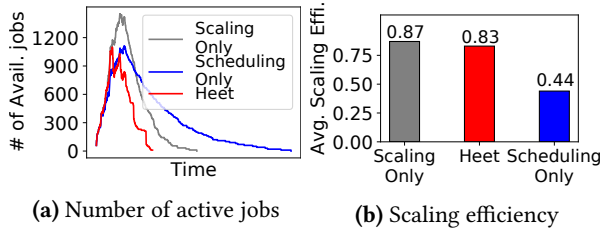


Figure 12. The advantage of balancing *scaling efficiency* and *scheduling efficiency*.

It is worth noting that our simulation testbed yields a negligible average margin of throughput error, measuring just 5%, resulting from dynamic resource contention on hosts over time. Within this simulated environment, we conducted an extensive investigation of the *scaling efficiency* and blocking index concerning the scheduling of 5000 jobs. This exploration encompassed two distinct scenarios of job arrivals: moderate contention and heavy contention. More precisely, the arrival rate in the heavy-contention setting was set at twice that of the moderate scenario.

Scaling efficiency. We first examined the *scaling efficiency* of jobs, as depicted in Fig. 11. It is evident that SRSF consistently exhibits the lowest and relatively stable *scaling efficiency* over time, regardless of whether the contention is moderate or heavy. Due to its fixed-worker resource allocation approach, Gavel showcases a *scaling efficiency* that solely depends on the characteristics of the jobs being processed, rendering it impervious to contention. Both Heet and AFS demonstrate a significant superiority over other baselines, primarily because they prioritize *scaling efficiency* to a great extent. Specifically, these schedulers have a tendency to assign a single worker to a higher number of jobs during periods of heavy contention, thereby naturally enhancing *scaling efficiency* for the majority of their scheduling time. It is worth noting that AFS, however, lacks the capability to effectively leverage resource heterogeneity, resulting in an imbalance of pending jobs across queues as time progresses. As a consequence, we observe that AFS falls behind Heet in terms of *scaling efficiency* over time, with a difference of 34.5%, even in heavy-contention scenarios.

Benefit of balancing scaling efficiency and scheduling efficiency. We investigated the advantages of adeptly balancing *scaling efficiency* and *scheduling efficiency* by conducting a comparative analysis involving Heet and the policies prioritizing *scaling efficiency* first and *scheduling efficiency* first. In Fig. 12(a), Heet is observed to maintain a slightly higher number of active jobs than the *scheduling efficiency*-only policy. However, it significantly diminishes the number of active jobs compared to the *scaling efficiency*-only policy during the early stage, where the presence of sojourning short jobs contributes to JCT inflation. In contrast, the *scaling efficiency*-only policy attains a remarkable 1.97 $\times$ higher *scaling efficiency* than the *scheduling efficiency*-only

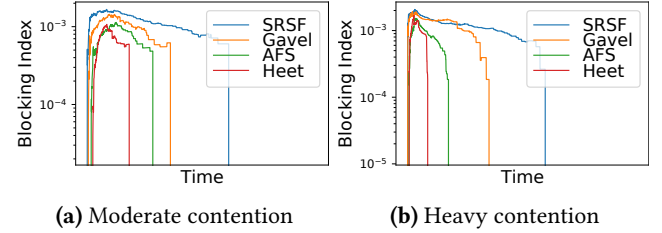


Figure 13. Job starvation across different contention levels.

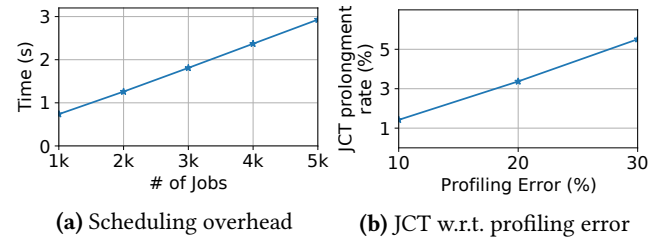


Figure 14. Efficiency and robustness of Heet.

policy, as depicted in Fig. 12(b). Notably, Heet experiences only a marginal *scaling efficiency* degradation of 4%. By skillfully navigating a trade-off between *scaling* and *scheduling efficiency*, Heet effectively mitigates overall JCT, achieving a reduction of up to 1.74 $\times$ compared to both the *scaling efficiency*-first and *scheduling efficiency*-first policies.

Effect of trading mechanism. Next, we performed an experiment to evaluate the efficacy of Heet's trading mechanism. We compared the overall JCT performance in two scenarios: one with trading and one without trading, utilizing traces under moderate and heavy contention, respectively. The findings indicate that the trading mechanism results in a notable reduction in the overall JCT by up to 1.08 $\times$.

Starvation behavior. We also assessed the starvation behavior of Heet by measuring the blocking index among jobs, as illustrated in Fig. 13. As expected, SRSF induces the most significant head-of-line (HOL) blocking in both moderate and heavy contention scenarios, resulting in the highest index value throughout the duration of the experiment. Likewise, Gavel exhibits elevated index values due to its limited ability to adapt worker sizes, resulting in a higher number of pending jobs compared to the more flexible schedulers, namely AFS and Heet. However, it is worth noting that Heet, equipped with the capability to leverage cluster heterogeneity, successfully mitigates the queue imbalances. Consequently, in the case of congested queues, Heet efficiently schedules short jobs in alternative locations, effectively reducing the blocking index by 40.5% compared to AFS in heavy-contention scenarios.

Scheduling scalability. To quantify the scheduling complexity of Heet, we meticulously evaluated the overhead associated with the matching process in an emulated environment consisting of seven unique machine types, each

equipped with 100 GPUs. As depicted in Fig. 14(a), even when making scheduling decisions for a substantial workload of 5000 jobs, the incurred overhead remains minimal, amounting to only 2.93 seconds. This finding demonstrates the exceptional scalability of Heet in large-scale cluster.

Performance under inaccurate profiling. Considering the inevitable occurrence of profiling error in Heet, we conducted an analysis to measure its impact on JCT under different levels of profiling error, as shown in Fig. 14(b). Specifically, we observed that when the estimated job throughput deviates by up to $\pm 30\%$, the overall JCT experienced a prolongation of 5.5%. This result highlights Heet's resilience to profiling errors. Furthermore, in our large-scale simulation testbed, Heet consistently outperformed AFS-L, Gavel, and SRSF, achieving JCT reduction rates of 1.23 $\times$, 1.84 $\times$, and 2.4 $\times$, respectively, even under a profiling error of up to 30%.

7 Related Works

Homogeneous DL schedulers. The majority of research for scheduling DL jobs has focused on homogeneous GPUs. Gandiva [51] and Lucid [21] concentrate on packing DL jobs into GPUs to enhance cluster efficiency. Zeus [54], HiveD [55], Chronus [14], Elasticflow [16], Lyra [29], Tiresias [17], and AFS [24] optimize their scheduling strategies towards specific objectives, such as reducing JCT, minimizing energy consumption, or meeting service level objectives.

Heterogeneous DL schedulers. Existing schedulers also strive to offer better JCT or fairness performance for non-elastic DL jobs in heterogeneous GPU clusters. Gavel [36] and AlloX [27] leverage user-specified information to optimize overall JCT. Gandiva_{fair} [8] co-optimizes fairness and efficiency within this context. HeteroG [53] specifically addresses the improvement of inter-worker communication. Notably, adapting these heterogeneity-aware schedulers to the elastic setting proves impractical as they allocate resources based on training throughput or completion time, which can vary substantially across elastic configurations. Integrating this variable information into their optimization framework is unfeasible.

Elastic DL schedulers. Various elastic schedulers have been proposed to improve resource utilization in homogeneous DL clusters. In particular, AFS [24], Elasticflow [16], Lyra [29], and Themis [32] are designed to improve JCT without requiring modifications to hyperparameters. On the other hand, Pollux [42] can adaptively adjust batch sizes among jobs to expedite their completion. In comparison to these schedulers, Heet focuses on investigating the specific influence of heterogeneity on scaling efficiency, a critical factor for optimizing resource allocation. Additionally, Heet successfully achieves a delicate balance between scaling and scheduling efficiency in heterogeneous environments.

8 Discussion

Support various parallelism paradigms in Heet. The current implementation of Heet primarily supports data parallelism, eliminating the need for alterations to user code when employing varying numbers of GPUs. Despite this focus, Heet is also adept at accommodating model-parallelism-based techniques such as tensor [37, 46] and pipeline parallelism [13, 22, 35]. This versatility arises from the fact that training paradigm changes don't disrupt the evaluation process of job length and scaling efficiency, the pivotal aspects of Heet's optimization framework. This adaptability equips Heet to effectively support large-scale model training, extending its utility to more parallelization strategies.

The Limitation of Heet. While Heet is a highly effective scheduler, it does have certain limitations in specific corner cases. For instance, when all jobs exhibit identically high scaling efficiency but possess a significant length gap in a resource-intensive cluster, Heet offers each job a single GPU. However, this approach may not be optimal in such cases, as providing some jobs with additional GPUs could expedite their completion and prevent JCT inflation. Nonetheless, such scenarios are relatively rare, particularly as model sizes increase. As models become larger, communication becomes a significant bottleneck that cannot be fully overlapped by computation, thereby reducing the scaling efficiency of models. Moreover, as Heet does not change the user-specified hyper-parameters, it cannot offer a more fine-grained performance improvement such as goodput optimization [42].

9 Conclusion

In this paper, we investigate a previously overlooked problem in the literature regarding the impact of resource heterogeneity on *scaling efficiency* in distributed DL training. To minimize the overall job completion time, we propose Heet, a new heterogeneity-aware system specifically designed for elastic training in DL clusters, focusing on two critical challenges. First, Heet employs machine learning techniques to accurately estimate the cost-efficient elastic training configuration in the presence of heterogeneity. Second, Heet incorporates a price function that strives to strike a balance between scaling and *scheduling efficiency*. Heet also integrates a scalable mechanism utilizing minimum-weight full matching and opportunistic resource trading to generate dynamic scheduling decisions.

Acknowledgments

We thank the anonymous reviewers, and our shepherd, Prof. Ali R. Butt for their helpful comments. This work is supported in part by the Science and Technology Development Fund of Macau (0024/2022/A1, 0071/2023/ITP2, 0081/2022/A2, and 0123/2022/AFJ), the Multi-Year Research Grant of University of Macau (MYRG2022-00119-FST, MYRG-GRG2023-00019-FST-UMDF).

A Artifact Appendix

A.1 Abstract

We present Heet, a Heterogeneity-aware system explicitly developed for elastic training in DL clusters. To demonstrate the superiority of Heet, we release our artifact in Zenodo. This artifact contains the source code for the proposed Heet's implementation. To be specific, we incorporate all the components (e.g., scheduler and agent) and functionalities (e.g., profiler, orchestrator, and placement optimizer) discussed in the paper. Moreover, we provide users with example training and profiling scripts, where users can simply replace it with their own training models and datasets. Example configuration and explanation can be found in our codes.

A.2 Hosting

The source code of Heet is publicly available on Zenodo: <https://zenodo.org/records/10512362>.

A.3 Hardware Configurations

Heet assumes that you have multiple types of GPUs. In our experiment, we use GPUs of three types, i.e., T4, A10, and V100. For each GPU type, we have two 4-worker GPU hosts.

A.4 Software Dependencies

We list the software dependencies required by building up the Heet system in a cluster as follows:

- Python
- rSync
- Pytorch
- Flask
- surprise
- networkX
- Horovod
- Redis

References

- [1] Alibaba cloud. <https://www.alibabacloud.com/product/gpu>, 2023.
- [2] Funk svd. <https://sifter.org/~simon/journal/20061211.html>, 2023.
- [3] Google cloud. <https://cloud.google.com/gpu>, 2023.
- [4] Nccl: accelerated multi-gpu collective communication. <https://images.nvidia.com/events/sc15/pdfs/NCCL-Woolley.pdf>, 2023.
- [5] Technical Report. <https://www.dropbox.com/scl/fi/kpzudtyls285lp3zhzgvg/Tech.pdf?rlkey=glhq3rpaipaizaa1k04bl67t7&dl=0>, 2023.
- [6] Robert M Bell, Yehuda Koren, and Chris Volinsky. The bellkor 2008 solution to the netflix prize. *Statistics Research Department at AT&T Research*, 1(1), 2008.
- [7] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics Paris France, August 22-27, 2010 Keynote, Invited and Contributed Papers*, pages 177–186. Springer, 2010.
- [8] Shubham Chaudhary, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, and Srinidhi Viswanatha. Balancing efficiency and fairness in heterogeneous gpu clusters for deep learning. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–16, 2020.
- [9] Chen Chen, Qizhen Weng, Wei Wang, Baochun Li, and Bo Li. Semi-dynamic load balancing: Efficient distributed learning in non-dedicated environments. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 431–446, 2020.
- [10] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. *ACM SIGPLAN Notices*, 48(4):77–88, 2013.
- [11] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. *ACM SIGPLAN Notices*, 49(4):127–144, 2014.
- [12] Christina Delimitrou and Christos Kozyrakis. Bolt: I know what you did last summer... in the cloud. *ACM SIGARCH Computer Architecture News*, 45(1):599–613, 2017.
- [13] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. Dapple: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 431–445, 2021.
- [14] Wei Gao, Zhisheng Ye, Peng Sun, Yonggang Wen, and Tianwei Zhang. Chronus: A novel deadline-aware scheduler for deep learning training jobs. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 609–623, 2021.
- [15] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. In *Proceedings of ACM SIGCOMM*, August 2014.
- [16] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. Elasticflow: An elastic serverless training platform for distributed deep learning. 2023.
- [17] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Harry Liu, and Chuanxiong Guo. Tiresias: A gpu cluster manager for distributed deep learning. In *NSDI*, volume 19, pages 485–500, 2019.
- [18] Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.
- [19] Jonathan L Herlocker, Joseph A Konstan, and John Riedl. Explaining collaborative filtering recommendations. In *Proceedings of the 2000 ACM conference on Computer supported cooperative work*, pages 241–250, 2000.
- [20] Qinghao Hu, Peng Sun, Shengen Yan, Yonggang Wen, and Tianwei Zhang. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of SC*, 2021.
- [21] Qinghao Hu, Meng Zhang, Peng Sun, Yonggang Wen, and Tianwei Zhang. Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 457–472, 2023.
- [22] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [23] Nicolas Hug. Surprise: A python library for recommender systems. *Journal of Open Source Software*, 5(52):2174, 2020.
- [24] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and Kyoungsoo Park. Elastic resource sharing for distributed deep learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 721–739, 2021.
- [25] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *Proceedings of ATC*, 2019.

- [26] Richard M Karp. An algorithm to solve the $m \times n$ assignment problem in expected time $O(mn \log n)$. *Networks*, 10(2):143–152, 1980.
- [27] Tan N Le, Xiao Sun, Mosharaf Chowdhury, and Zhenhua Liu. Allox: compute allocation in hybrid clusters. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–16, 2020.
- [28] Ang Li, Shuaiwen Leon Song, Jieyang Chen, Jiajia Li, Xu Liu, Nathan R Tallent, and Kevin J Barker. Evaluating modern gpu interconnect: Pcie, nvlink, nv-sli, nvswitch and gpudirect. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):94–110, 2019.
- [29] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages 835–850, 2023.
- [30] Qinyi Luo, Jiaao He, Youwei Zhuo, and Xuehai Qian. Prague: High-performance heterogeneity-aware asynchronous decentralized training. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 401–416, 2020.
- [31] Qinyi Luo, Jinkun Lin, Youwei Zhuo, and Xuehai Qian. Hop: Heterogeneity-aware decentralized training. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 893–907, 2019.
- [32] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient gpu cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation*, 2020.
- [33] Luo Mai, Guo Li, Marcel Wagenländer, Konstantinos Fertakis, Andrei-Octavian Brabete, and Peter Pietzuch. Kungfu: Making training in distributed machine learning adaptive. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, pages 937–954, 2020.
- [34] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond gpus for dnn scheduling on multi-tenant clusters. In *Proceedings of OSDI*, July 2022.
- [35] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 1–15, 2019.
- [36] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, pages 481–498, 2020.
- [37] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.
- [38] Andrew Or, Haoyu Zhang, and Michael Freedman. Resource elasticity in distributed deep learning. *Proceedings of Machine Learning and Systems*, 2:400–411, 2020.
- [39] Andrew Or, Haoyu Zhang, and Michael None Freedman. Virtualflow: Decoupling deep learning models from the underlying hardware. *Proceedings of Machine Learning and Systems*, 4:126–140, 2022.
- [40] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [41] Aurick Qiao, Abutalib Aghayev, Weiren Yu, Haoyang Chen, Qirong Ho, Garth A Gibson, and Eric P Xing. Litz: Elastic framework for high-performance distributed machine learning. In *2018 {USENIX} Annual Technical Conference ({USENIX} {ATC} 18)*, pages 631–644, 2018.
- [42] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *OSDI*, volume 21, pages 1–18, 2021.
- [43] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. Enabling compute-communication overlap in distributed deep learning training platforms. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 540–553. IEEE, 2021.
- [44] Herbert Robbins. A remark on stirling’s formula. *The American Mathematical Monthly*, 10(1):26–29, 1955.
- [45] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.
- [46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [47] Xiaoyuan Su and Taghi M Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in artificial intelligence*, 2009, 2009.
- [48] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 93–106, 2022.
- [49] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous gpu clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960. USENIX Association, 2022.
- [50] Ian H Witten and Eibe Frank. Data mining: practical machine learning tools and techniques with java implementations. *Acm Sigmod Record*, 31(1):76–77, 2002.
- [51] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pages 595–610, 2018.
- [52] Lei Xie, Jidong Zhai, Baodong Wu, Yuanbo Wang, Xingcheng Zhang, Peng Sun, and Shengen Yan. Elan: Towards generic and efficient elastic training for deep learning. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, pages 78–88. IEEE, 2020.
- [53] Xiaodong Yi, Shiwei Zhang, Ziyue Luo, Guoping Long, Lansong Diao, Chuan Wu, Zhen Zheng, Jun Yang, and Wei Lin. Optimizing distributed training deployment in heterogeneous gpu clusters. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, pages 93–107, 2020.
- [54] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. Zeus: Understanding and optimizing {GPU} energy consumption of {DNN} training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 119–139, 2023.
- [55] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis CM Lau, Yuqi Wang, Yifan Xiong, et al. Hived: Sharing a gpu cluster for deep learning with guarantees. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, pages 515–532, 2020.